Unveiling Malware: Static and Dynamic Analysis

This presentation explores the critical techniques of static and dynamic malware analysis. We will delve into how these methods help identify and understand malicious software.

by Buddy Knowledge Saathi

Understanding Static Analysis

Static analysis examines malware without executing it. This non-invasive approach provides initial insights into the sample's potential behavior and characteristics.

Safe Exploration

Analyze code without risk. Identify threats before they run.

Early Indicators

Spot suspicious elements. Uncover embedded clues.

Initial Assessment

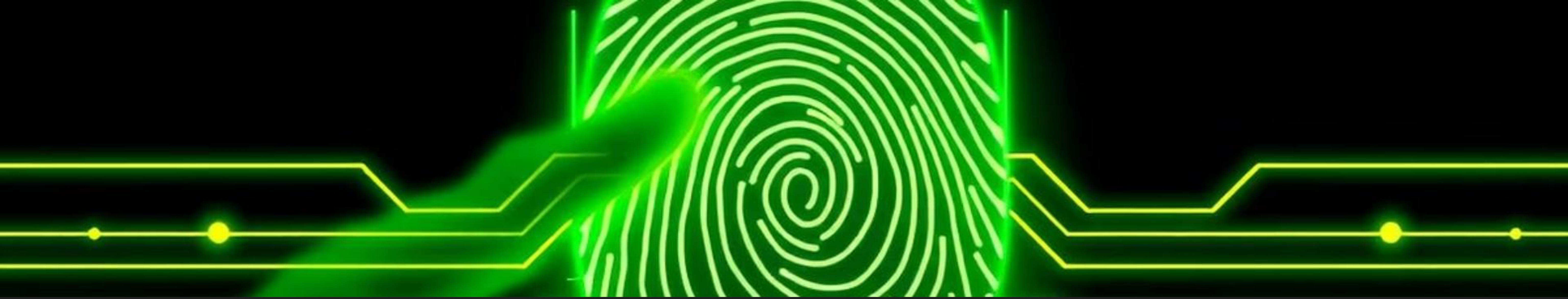
Formulate a preliminary understanding. Guide further investigation.

Hashing for Malware Identification

Hashing converts data into a fixed-size string. It ensures data integrity and helps identify known malware variants quickly.

MD5	128-bit	Fast, weak security (malware classification)
SHA-1	160-bit	Better than MD5 but deprecated
SHA-256	256-bit	Strong integrity, industry standard

These algorithms create unique fingerprints for files. They are crucial for initial triage.



Purpose of Hashing in Malware Analysis

Hashing serves multiple vital purposes in static malware analysis. It helps in rapid identification and cross-referencing.

File Identification

Quickly determine if a file is a known threat.

Integrity Verification

Confirm if a file has been tampered with or corrupted.

Database Lookup

Check hashes against large malware databases like VirusTotal.

Malware Classification

Categorize samples based on known malicious hash sets.

Essential Hashing Tools

Analysts use various tools to generate and compare file hashes. These tools streamline the identification process.

Command Line

Utilize **md5sum** or **sha256sum** for quick hash generation.

GUI Tools

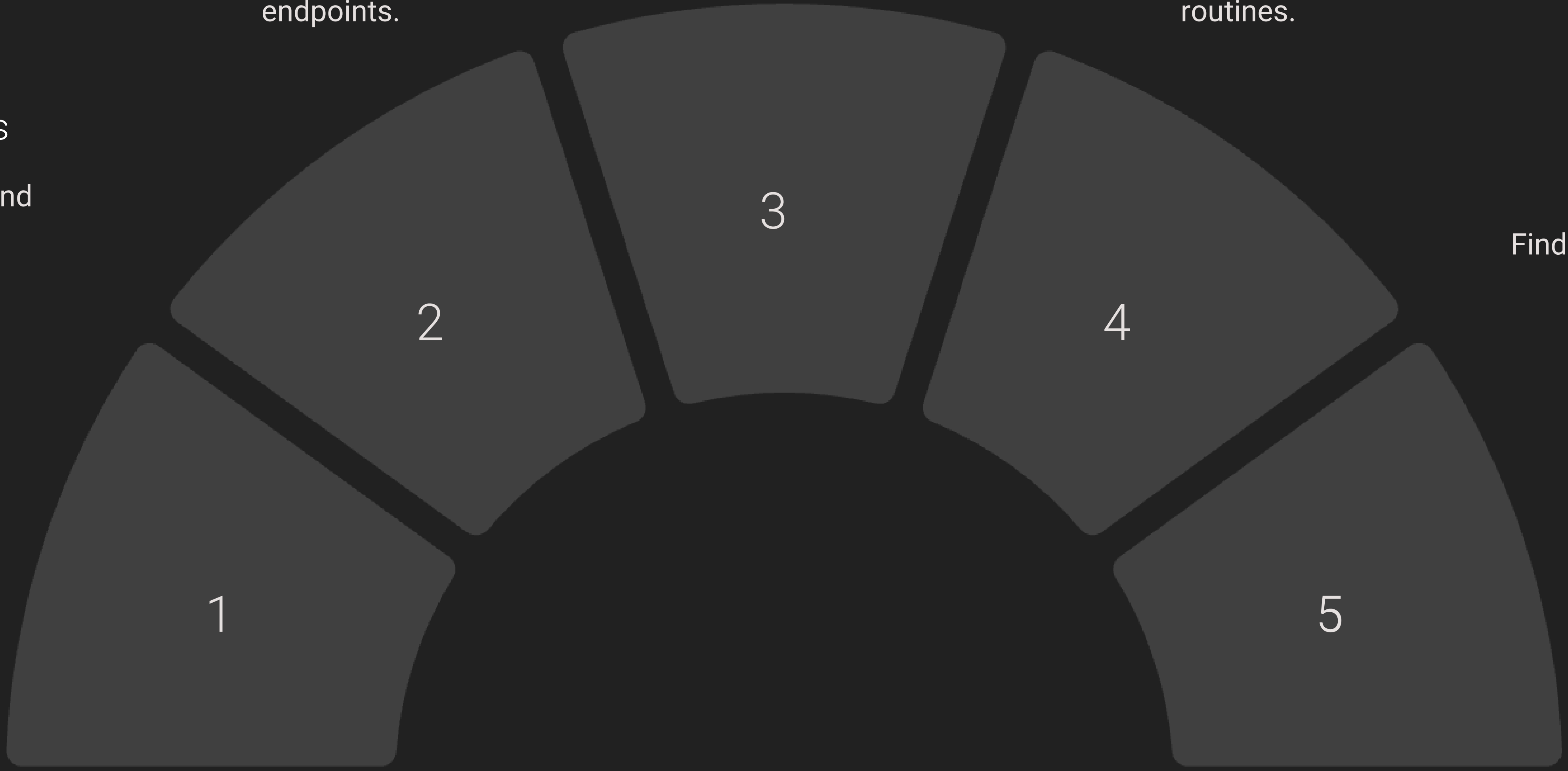
HashMyFiles offers a user-friendly graphical interface.

Forensic Suites

FTK Imager and Autopsy integrate hashing into broader analysis.

Finding Strings in Binaries

Identifying human-readable strings within binary files reveals crucial information. These strings can expose hidden functionalities.



Reveals URLs

Discover command and control servers.

Exposes IPs

Identify communication endpoints.

Hardcoded Creds

Uncover embedded usernames and passwords.

Function Names

Identify internal program routines.

Commands

Find execution instructions.

Tools for String Extraction

Specialized tools facilitate the extraction of strings from malware samples. They help in quickly pinpointing suspicious text.

Strings Utility

The classic command-line tool **strings malware.exe** is often the first step for quick string analysis.

BinText

A popular graphical tool providing advanced filtering and search capabilities for strings in executable files.

Flare VM

This virtual machine includes many pre-installed tools optimized for malware analysis, including string utilities.

PEStudio

Offers a dedicated 'Strings' tab for easy viewing and analysis of embedded strings in Portable Executables.

The Art of Static Analysis

Static analysis combines various techniques to build an initial understanding of malware. It's a foundational step in any investigation.

Initial Triage

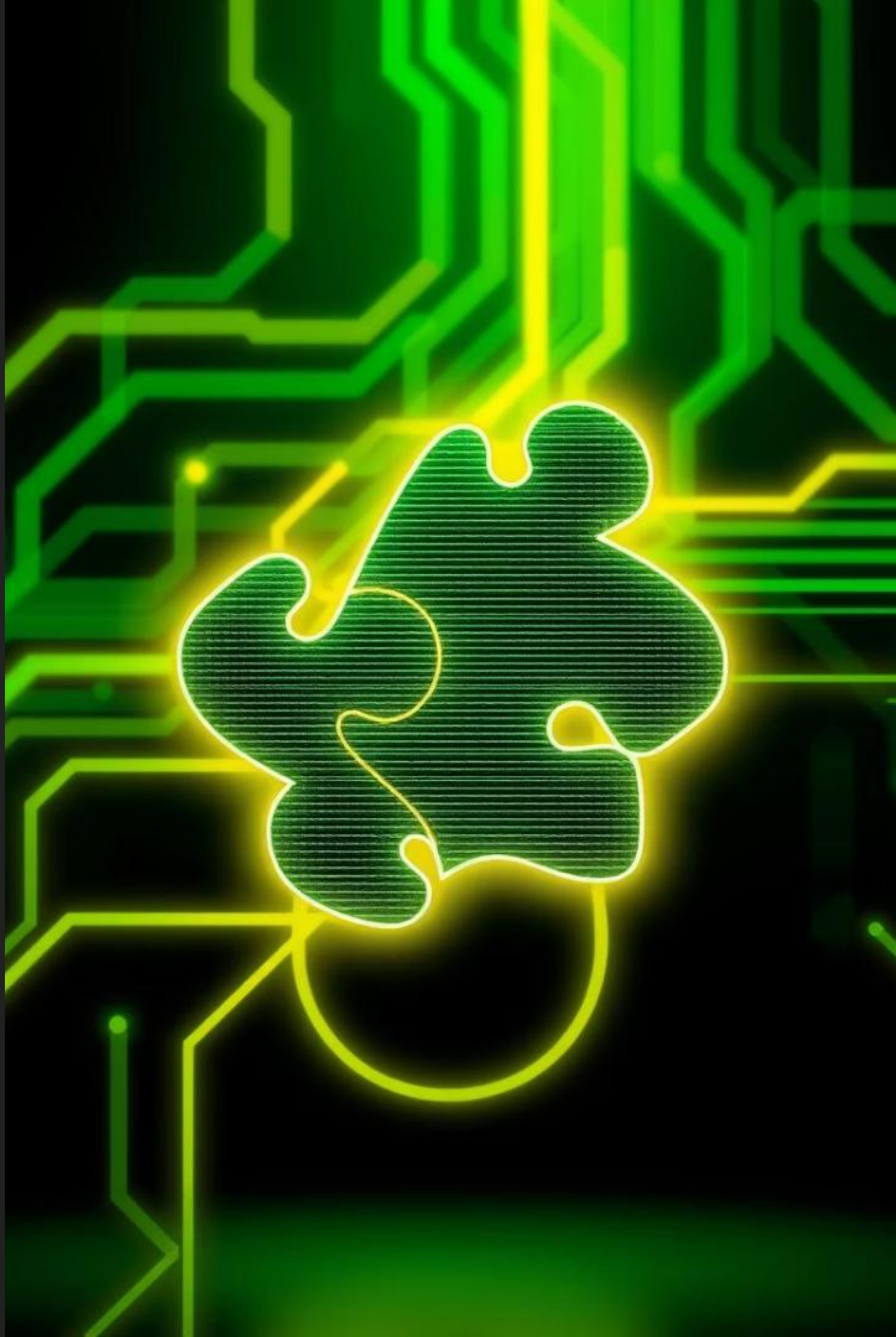
Quickly assess risk and gather high-level indicators.

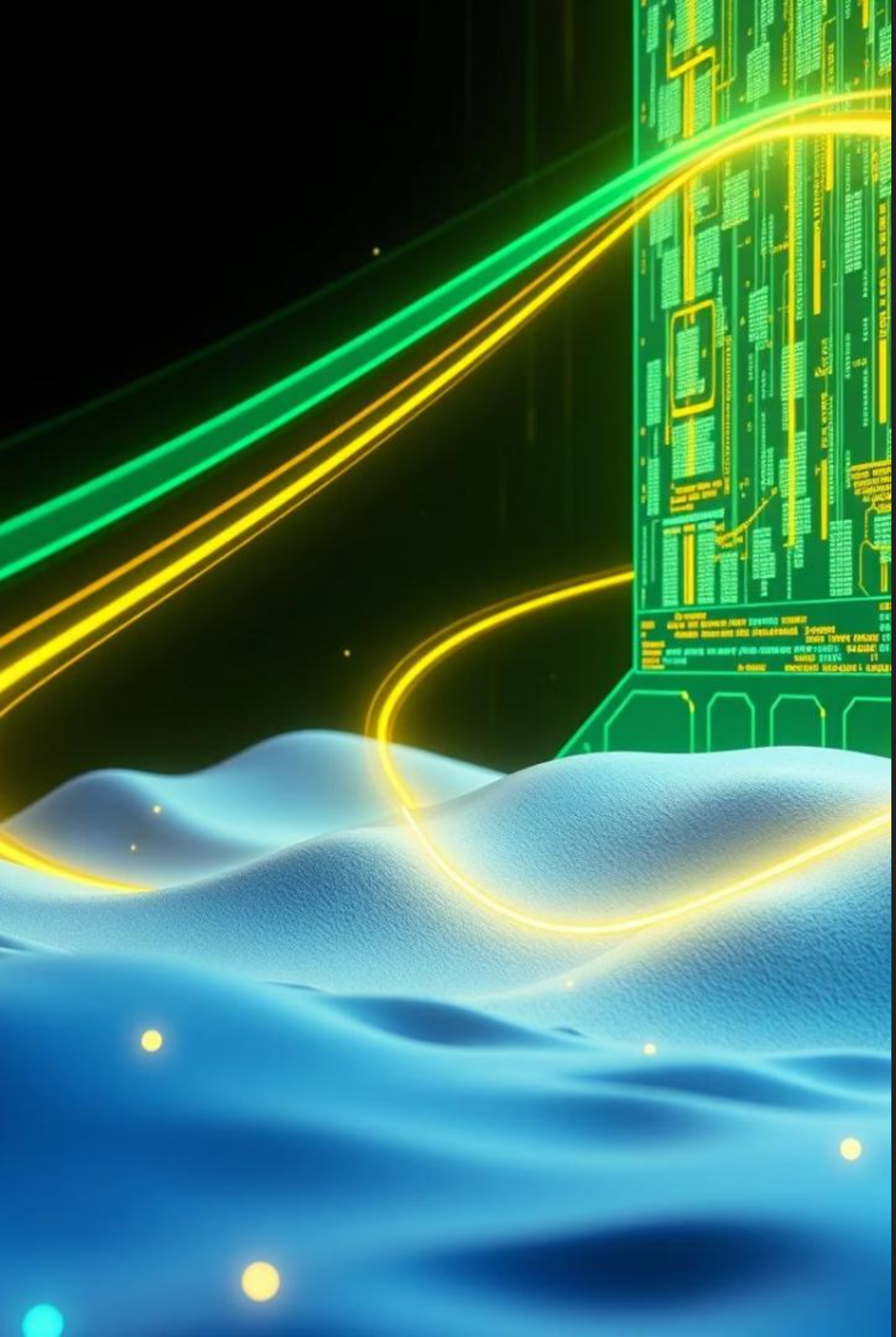
Behavior Prediction

Infer potential actions without execution.

Targeted Dynamic Analysis

Inform and prepare for deeper, active investigation.





Transition to Dynamic Analysis

While static analysis provides a blueprint, dynamic analysis confirms behaviors. It involves executing the malware in a controlled environment.

This next phase validates hypotheses and exposes runtime interactions.

Static Insights

Examine code, identify strings, calculate hashes.

Execute Safely

Run malware in a sandbox to observe actions.

Observe Behavior

Monitor network, file system, and process changes.

Confirm Threats

Validate malicious intent and impact.

Key Takeaways and Next Steps

Static analysis provides essential initial insights. Hashing and string extraction are fundamental techniques.

Dynamic analysis is crucial for behavioral confirmation. Mastering both methods is vital for comprehensive malware analysis.

2	3	1
Analysis Types	Static Pillars	Combined Approach
Static for code review, Dynamic for behavior.	Hashing, strings, and metadata analysis.	Use both for a full picture.

Mastering Malware Analysis: Static & Dynamic Techniques

Explore the fundamental techniques of static and dynamic analysis to dissect and understand malicious software. This unit delves into the structure of Windows executables and how to identify suspicious behaviors.

by Buddy Knowledge Saathi

Understanding Portable Executable (PE) Files

Structure

PE is the standard format for Windows executables.

- Executables (.exe)
- Dynamic-link libraries (.dll)
- Drivers (.sys)

Key Components

Each section plays a critical role in file execution.

- **DOS Header:** Legacy compatibility
- **PE Header:** File metadata
- **.text:** Executable code
- **.data:** Initialized data
- **.rdata:** Read-only data
- **.rsrc:** Resources

Tools for PE File Analysis

PEStudio

Comprehensive static analysis tool.

CFF Explorer

Advanced PE editor and viewer.

Exeinfo PE

Identifies packer, compiler, or obfuscator.

Detect It Easy (DIE)

Detects file types and packers.

Analyzing Linked Libraries and Functions

DLLs Imported

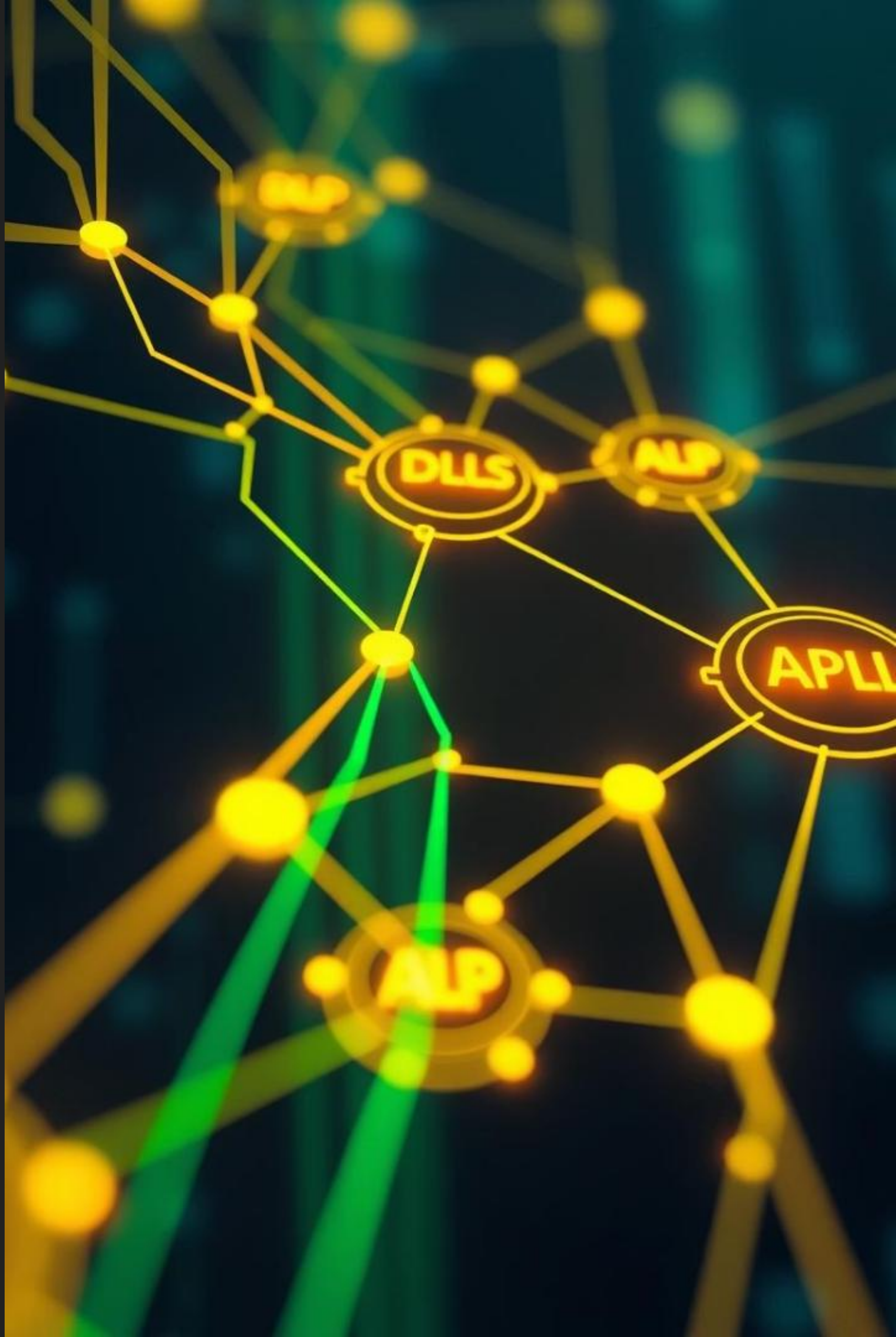
Examine kernel32.dll, user32.dll for system interactions.

API Calls

Look for CreateRemoteThread(), WinExec(), VirtualAlloc().

Suspicious Activity

Indicates potential malware or persistence methods.



Tools for Linked Libraries Analysis

Dependency Walker

Lists all dependent modules and functions.

PEStudio (Imports tab)

Provides quick view of imported functions.

IDA Free / Ghidra

Disassemblers for detailed code review.

Understanding Static Analysis Techniques

Code Review

Examine source or disassembled code.

Signature Matching

Identify known malware signatures.

String Analysis

Extract meaningful strings from binaries.

Key Indicators of Malware

1

Unusual Imports

DLLs for obscure system functions.

2

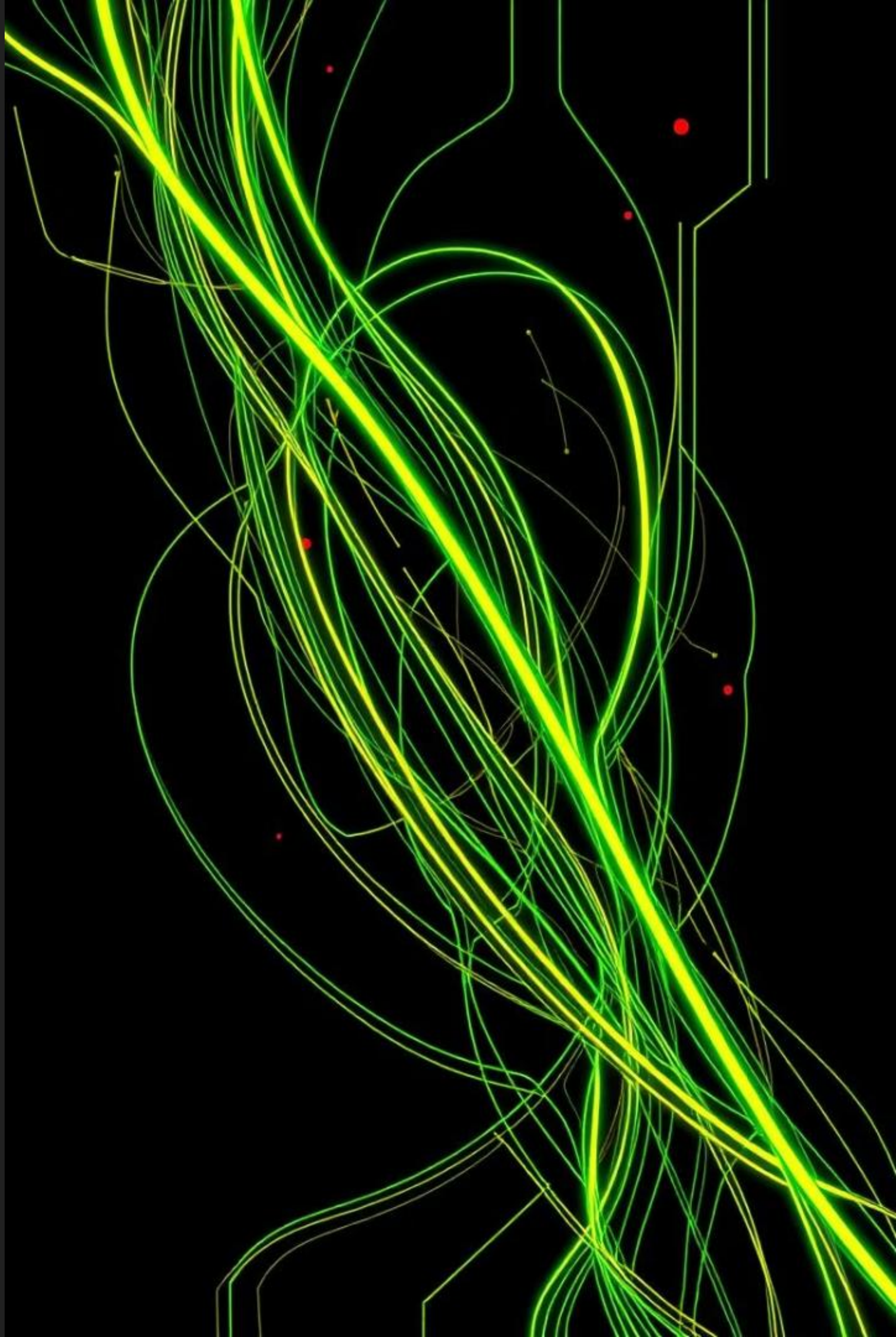
Packing/Obfuscation

Code hidden to evade detection.

3

Anti-Analysis

Techniques to detect debuggers.



Exploring Dynamic Analysis

Execute Malware

Run sample in a controlled environment.

Observe Behavior

Monitor network, file system changes.

Record Data

Log system calls, process activity.

Report Findings

Document all observed actions.

Dynamic Analysis Tools

Sandboxie

Isolates programs from the system.

Wireshark

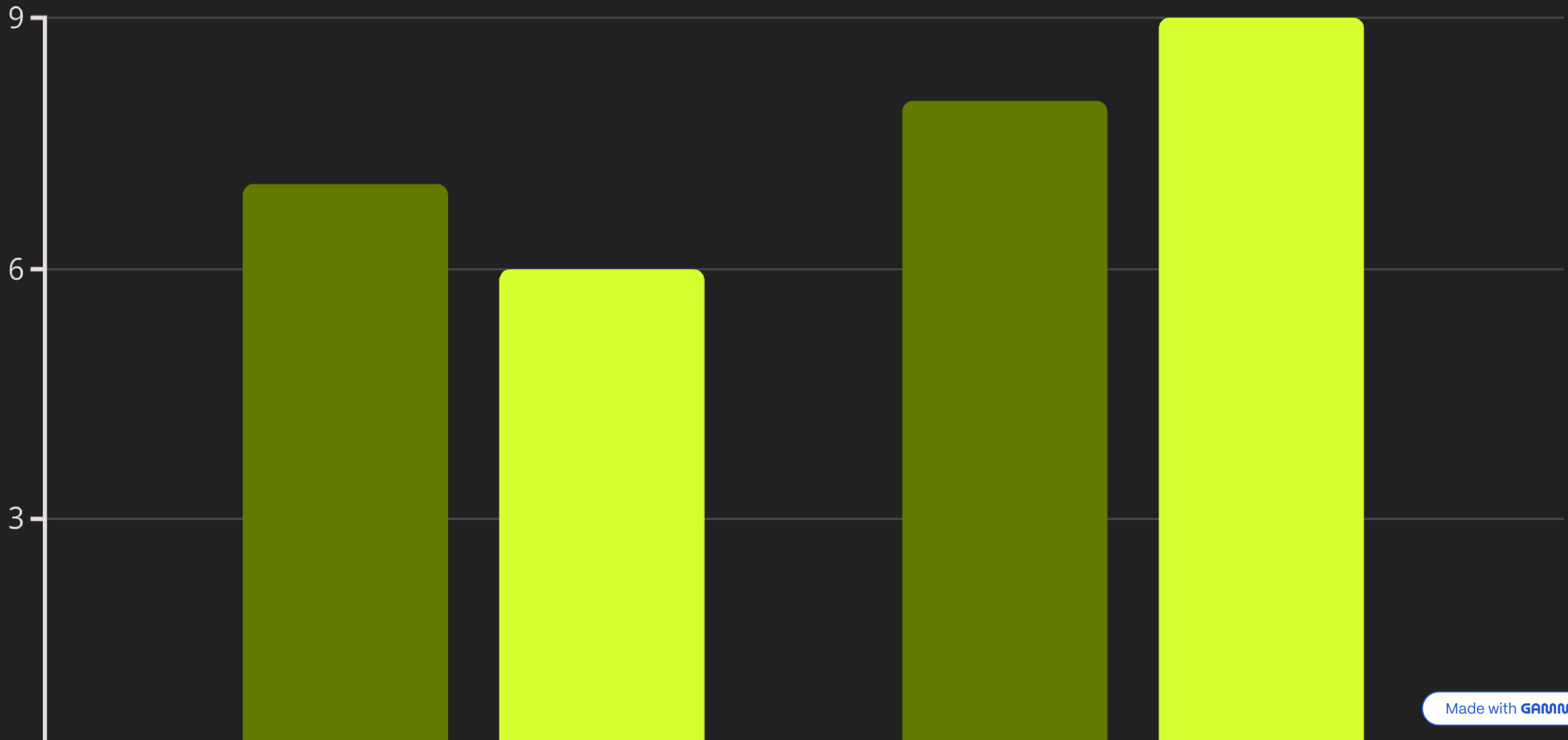
Captures and analyzes network traffic.

Process Monitor

Monitors file system, registry, process activity.

Conclusion: Static vs. Dynamic Analysis

Both static and dynamic analysis are crucial for a complete understanding of malware. Static analysis provides initial insights into the malware's structure and potential capabilities without execution. Dynamic analysis reveals the malware's actual behavior in a controlled environment, confirming suspicions and uncovering hidden functionalities.



Mastering Malware Analysis

This presentation explores the critical techniques of static and dynamic malware analysis. We will delve into identifying malicious indicators and utilizing essential tools.

by Buddy Knowledge Saathi

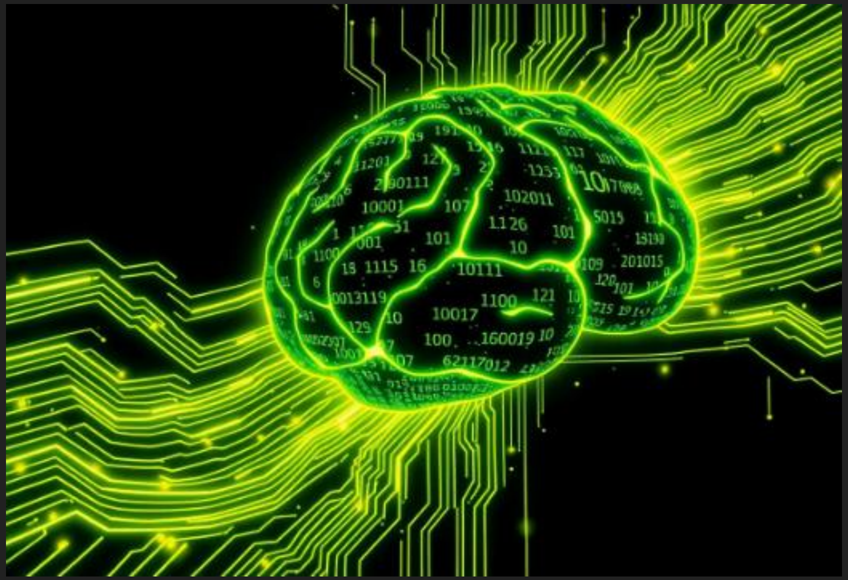
Understanding Linked Libraries and Functions

What to Analyze

- DLLs imported: **kernel32.dll, user32.dll**
- API calls: **CreateRemoteThread(), WinExec()**

Indicators

- Suspicious DLLs and APIs suggest malicious activity.
- System-level calls indicate injection or persistence.



Essential Tools for Library Analysis

Dependency Walker

Visualizes imported DLLs and functions.

PEStudio

Provides detailed import analysis.

IDA Free / Ghidra

Powerful disassemblers for deep dives.

These tools are crucial for static analysis, helping to identify suspicious connections and hidden functionalities within executables.

The Role of Virtual Machines in Malware Analysis

Safe Environment

VMs provide isolation for execution.

Snapshots

Rollback to clean states easily.

Simulated OS

Replicate real operating systems.

Key Virtualization Tools

VirtualBox

Open-source and versatile for various OS.

- Easy setup
- Cross-platform

VMware Workstation

Robust for professional analysis.

- Advanced features
- High performance

KVM

Linux-native hypervisor for performance.

- Integrated with Linux
- Hardware acceleration

Choosing the right VM platform depends on your specific analysis needs and host operating system.

Best Practices for VM Setup

Disable Shares

Turn off shared folders and clipboard.

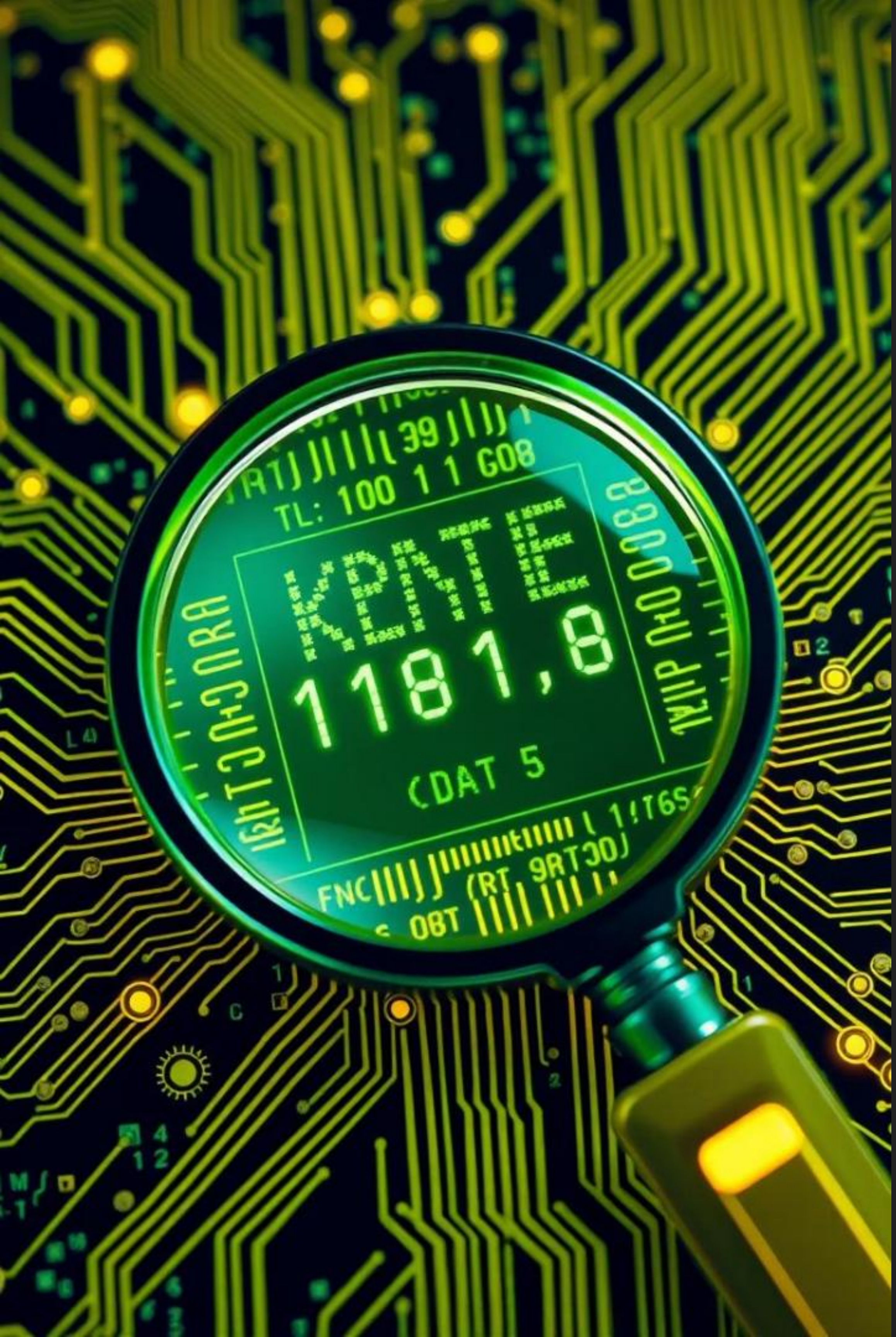
Limit Network

Only enable internet when essential.

Multiple OS

Use XP, 7, 10, and Linux variants.

These precautions prevent malware escape and ensure a controlled environment for observation.



From Static to Dynamic: The Malware Analysis Journey

Initial Static Analysis

Examine code without execution.

Controlled Dynamic Execution

Run malware in a safe VM.

Behavioral Monitoring

Observe file system, network, and registry.

Reporting & Mitigation

Document findings and devise countermeasures.

Stages of Malware Analysis

Static Analysis

Initial code inspection and string extraction.

Dynamic Analysis

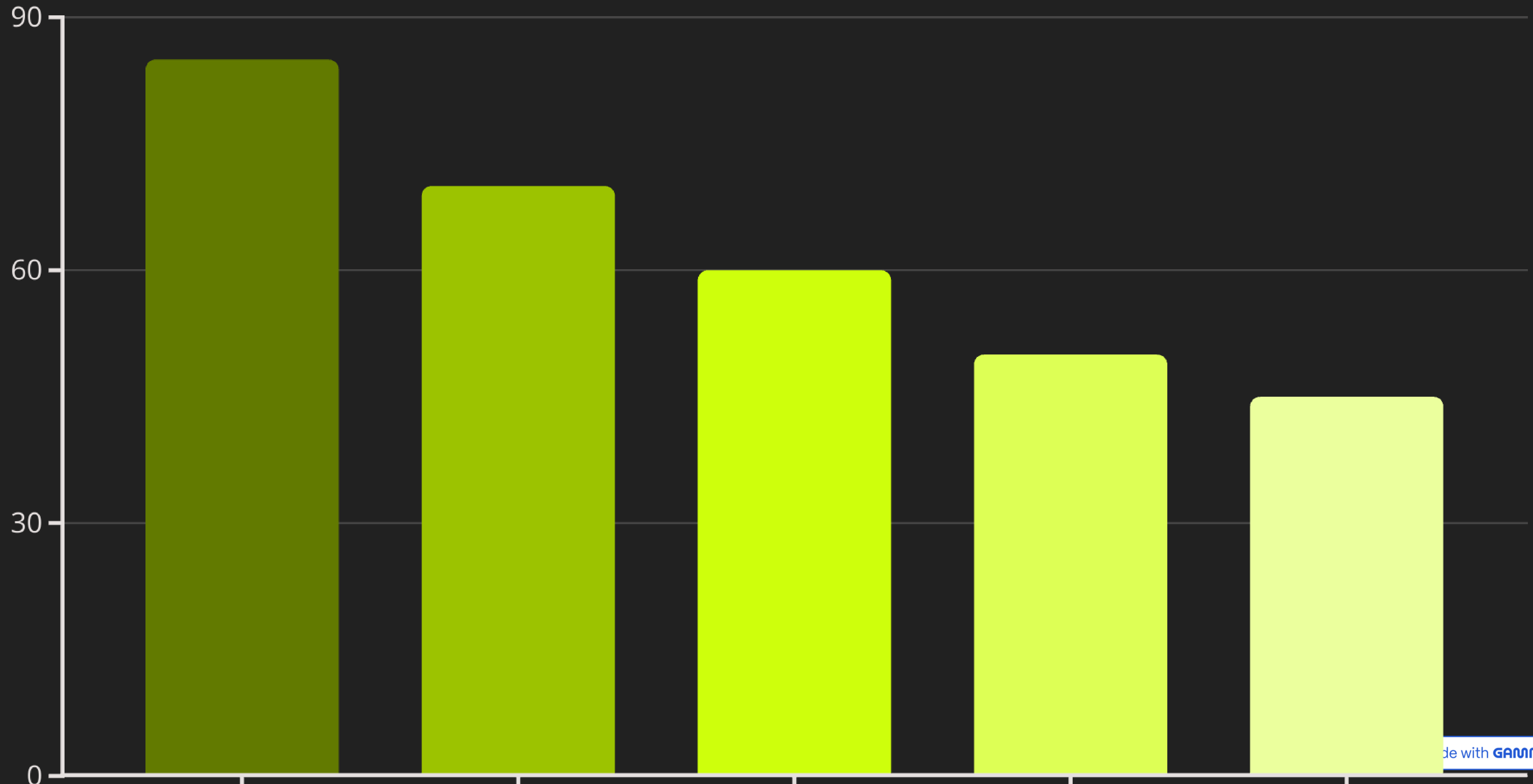
Observe runtime behavior in isolation.

Behavioral Analysis

Detailed monitoring of system interactions.

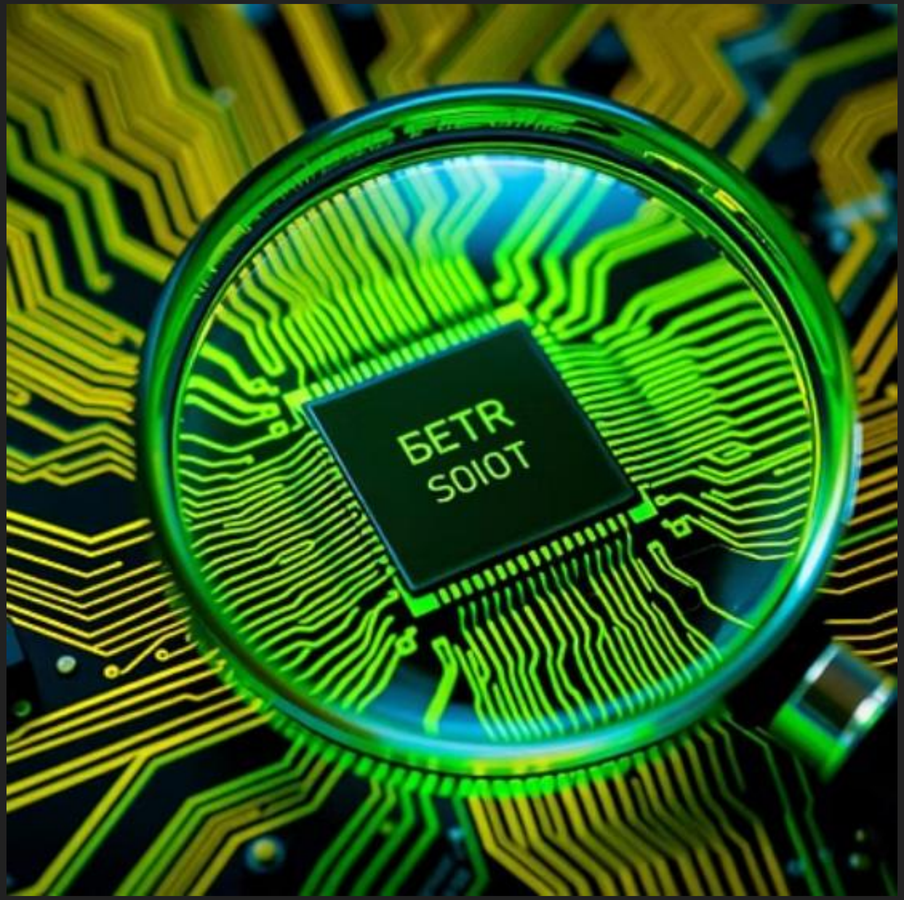
Each stage builds upon the previous, providing a comprehensive understanding of the malware's capabilities.

Indicators of Compromise



Key Takeaways and Next Steps

Static and dynamic analysis are foundational for understanding malware. Virtual machines offer safe environments for execution and observation. Always use a variety of tools for comprehensive insights.



Continue practicing these techniques to enhance your skills and contribute to a safer digital world.

Mastering Malware Analysis

This unit explores static and dynamic analysis techniques. Learn to safely dissect malware and understand its behavior. We will cover essential tools and methodologies.

by Buddy Knowledge Saathi

Dynamic Analysis: Behavioral Insights

Dynamic analysis observes malware execution in a controlled environment. It reveals runtime behaviors. This includes file system changes and network communications.

- Observe malware's actions in real-time.
- Understand its true intent and impact.
- Identify communication with command-and-control servers.

Automated Sandboxes for Malware

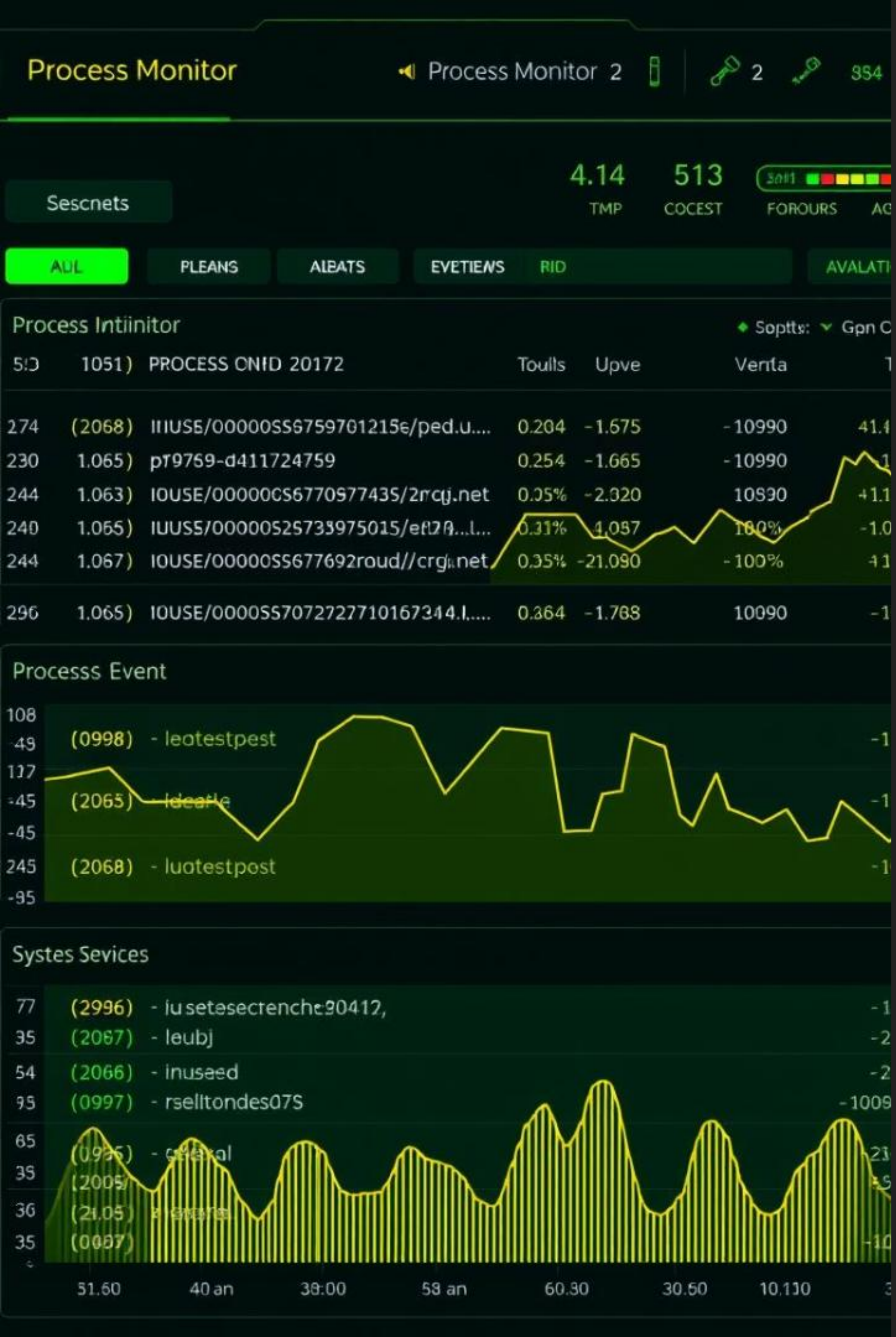
Sandboxes provide isolated, safe environments. They automatically execute malware samples. This allows for observation without risk to host systems.

Cuckoo Sandbox	Open-source, customizable, widely used for in-depth analysis.
Any.Run	Online, interactive malware analysis with cloud-based infrastructure.
Joe Sandbox	Deep malware inspection with behavior graphs and comprehensive reports.

Steps for Malware Execution and Monitoring

Execute malware safely using a structured process. This ensures isolation and comprehensive data capture. Reverting snapshots maintains a clean analysis environment.

1. ### Set Up Isolated VM
   Create a virtual machine without internet access.
2. ### Take a Snapshot
   Capture VM state before malware execution.
3. ### Start Monitoring Tools
   Launch RegShot, ProcMon, and Wireshark.
4. ### Run Malware Sample
   Execute the suspicious file.
5. ### Observe Behavior
   Monitor file, registry, and network changes.
6. ### Revert VM
   Restore the clean snapshot for next analysis.



ProcMon: Real-time System Insights

Process Monitor (ProcMon) offers real-time visibility. It tracks file, registry, and process activities. Filters help pinpoint suspicious behaviors.

File System Activity

Monitor file creation, deletion, and modification.

Registry Access

Track registry key reads, writes, and deletions.

Process/Thread Activity

Observe process launches and thread creations.

Network Operations

Capture network connection attempts by processes.

Process Explorer: Deep Dive into Processes

Process Explorer replaces Task Manager with advanced views. It displays process details, threads, and loaded DLLs. Crucial for detecting hidden or injected processes.

Key Features

- Parent-child process chains.
- DLLs loaded by each process.
- Open handles and active threads.

Use Cases

- Identify suspicious process injection.
- Detect hidden processes or malicious services.
- Analyze memory usage patterns.

RegShot: Registry Change Detection

RegShot compares registry states before and after execution. It generates a report detailing all modifications. Essential for identifying malware persistence mechanisms.

1. Take Shot 1

   Capture initial registry state before malware.

2. Run Malware Sample

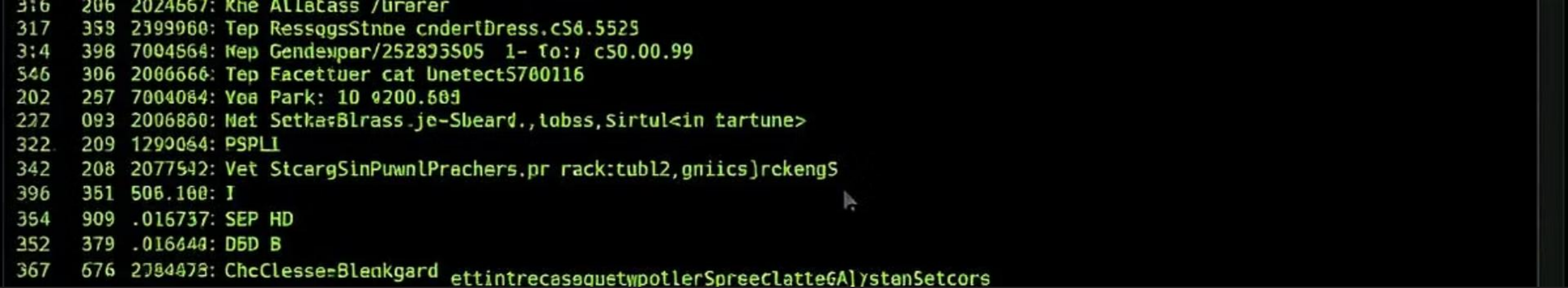
   Execute the suspicious file in the VM.

3. Take Shot 2

   Capture post-execution registry state.

4. Generate Diff Report

   Analyze changes to identify malicious modifications.



Wireshark for Network Packet Analysis

Wireshark captures and analyzes network traffic. It helps detect C2 beaconing and data exfiltration. Use filters to focus on relevant communications.

http	Detect HTTP communication protocols.
dns	Look up domain names requested by malware.
ip.addr == 192.168.1.100	Focus on a specific IP address.
tcp.stream eq 1	Follow a specific TCP connection stream.

Malware Analysis Toolset Overview

A diverse set of tools supports static and dynamic analysis. Each tool offers unique capabilities. Combined, they provide a comprehensive view of malware.

Static Analysis

- Strings: Finds readable text in binaries.
- PEStudio: Analyzes Portable Executable files.
- Ghidra/IDA Free: Disassembly and reverse engineering.

Dynamic Analysis

- ProcMon: File, registry, process activity.
- Process Explorer: Detailed process view.
- RegShot: Compares registry changes.
- Wireshark: Network traffic capture.

Automated Analysis

- Cuckoo Sandbox: Open-source automated analysis.
- Any.Run: Interactive online sandbox platform.

Conclusion: Advancing Your Analysis Skills

This unit has equipped you with critical malware analysis skills. You now understand both static and dynamic approaches. You can confidently investigate real-world malware threats.

1

Manual Dissection

Analyze and dissect malware step-by-step.

2

Dual Approach

Utilize both static and dynamic methods.

3

Behavioral Detection

Identify suspicious activities with system tools.

4

Safe Environments

Set up secure analysis infrastructures.